

Post Processing: Project Report

Authors: Atli Arnarsson, Drake Ma, Henry Morin, Andrew Petersen

Abstract:

The project's goal is to supply post-processing effects that can be freely applied to the ray tracer's finalized image buffer. Post-processing effects may use the camera and/or depth, normal, motion, and/or position maps generated during raytracing to generate the final image:

Depth map: A buffer where each element is the number of units travelled by the ray traced from the pixel, until it hits a shape.

Normal map: A buffer where each element is the 3D normal vector of the shape that was hit by the ray traced from the pixel.

Motion map: A buffer where each element is the 3D velocity vector of the shape that was hit by the ray traced from the pixel.

Position map: A buffer where each element is the 3D position vector of the shape that was hit by the ray traced from the pixel

Table of Contents

Pipelining	3
Additive/Multiplicative Blending	4
Overlay.....	5
Hard Light.....	6
Hue Shift	7
Tone Mapping	8
Ordered and Floyd-Steinberg Dithering	8
Motion Blur (DM)	10
Daltonization (DM)	13
Blur	16
Edge Detect	17
SSAO.....	18

Pipelining

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

Additive/Multiplicative Blending

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

Overlay

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

Hard Light

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

Hue Shift

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

Tone Mapping

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

Ordered and Floyd-Steinberg Dithering

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

Motion Blur (DM)

This effect applies bi-directional blur to objects in the scene as if they're in motion.

How to run

Go to `PostProcessor.h` and search for `DefaultPipeline` on line 62. Replace the content of the `buildPipeline()` method with the following:

```
return new PostProcessor(ConstMultiply(new RGBConvert(new ToneMapHSV(new  
HSVConvert((new MotionBlur(imageBuffer))->init(motionBuffer))), 255.0f));
```

To modify the velocity for certain objects in the scene, go to `ObjLoader.h`. To change the spheres' velocity, head to **line 31**. To change the triangles' velocity, head to **line 51**. To change the camera's velocity, head to **line 128** (4th parameter of the `Camera` constructor).

Accomplishments

This effect is able to accurately portray moving objects in the scene with appropriate blurs. This is done by performing the following steps:

1. Assign each shape with a velocity based on their material
2. For each ray that hits a shape: (`RayTracer.h`)
 - a. The `Hitpoint` receives velocity from the object
 - b. Calculate the final 3D motion by subtracting the camera velocity from the object velocity
 - c. Project the 3D motion in the direction of ray so that `x` and `y` are in terms of camera's 2D space
 - d. Negate `y` of the newly calculated 2D motion
 - e. Scale the 2D motion by the focal length of the camera and the depth of the ray (parameter)
 - f. Put the 2D motion inside the motion map
3. Dilate the motion map (inspired by [McGuire's 2012 paper](#)) (`MotionBuffer.h`)
 - a. Divides the motion map into chunks of 8x8 pixel size (customizable)
 - b. For each chunk, iterate over each pixel inside the chunk to get the motion with the highest magnitude
 - c. For each chunk, iterate over its neighboring chunks to get the motion with the highest magnitude
 - d. For each pixel in each chunk, sample 32 points (customizable) along the chunk motion and then average it to get the final motion for the pixel

4. Blur the image using the dilated motion map (`MotionBlur.h`)
 - a. For each pixel, sample 32 points (customizable) along the pixel motion in the image buffer and average these colors to get the final pixel color

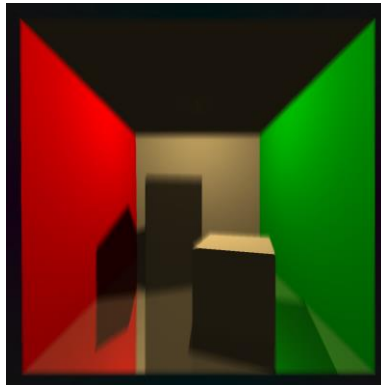
State-of-the-art

- Hybrid Rendering: MBlur
 - o When moving objects are covered by static foreground objects, there may be inaccuracies in how the blur is portrayed
 - o MBlur uses another pass of ray tracing to get data on each pixel to calculate the final blur needed

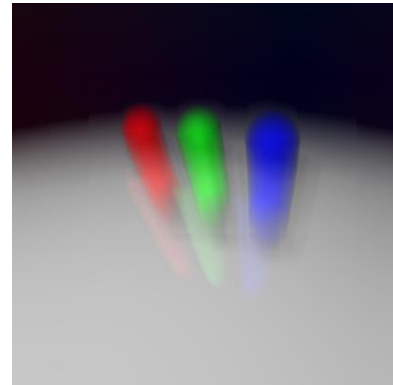
Results



Stanford bunny with a velocity of $\langle 0, 0, 1 \rangle$ (moving parallel to the camera's line of sight)



Cornell box, both the camera and the short box have velocity of $\langle 0, 50, 0 \rangle$



Spheres scene, camera moving **up** (we forgot what the velocity was)

What We Learned

Motion blur as a post process effect can be cheap to calculate but requires advanced techniques to match the accuracy of realistic motion blur. A more realistic motion blur can be done by tracing the scene multiple times, with each frame containing objects that move very slightly based on delta (time difference between each frame). However, that is very time consuming, so post process motion blur is used more commonly for real-time applications.

Next Steps

Currently, if a static foreground covers a moving object, the blend will bleed into the foreground object, which is not ideal. It may be better if we somehow compare the final motion map with the depth map and modify each motion based on depth differences. However, that is only a guess so it might not work out as intended. MBlur is also something worth looking into and would be useful to implement.

Other Details

N/A

Daltonization (DM)

This effect reduces color confusion for people with Color Vision Deficiency (CVD) by enhancing the color they can see clearly and reducing the color they often confuse with. This effect can also simulate CVD and the color enhanced image in their perspective.

How to run

Go to `PostProcessor.h` and search for `DefaultPipeline` on line **62**. Replace the content of the `buildPipeline()` method with the following:

```
CVDType cvd_type = CVDType::PROTANOPIA;
bool compensate = true;
bool simulate = false;
return new PostProcessor(ConstMultiply(new RGBConvert(new ToneMapHSV(new
HSVConvert((new Daltonization(imageBuffer))->init(cvd_type, compensate,
simulate))))) , 255.0f));
```

By default, `compensate` is set to `true` and `simulate` is set to `false` in the Daltonization initialization. However, they are explicitly defined here for demonstration purposes. There are 3 types of CVD implemented here: Protanopia, Deutanopia, and Tritanopia. If `compensate` is set to `true`, the image will be recolored to assist the person with the designated CVD. If `simulate` is set to `true`, the image will be recolored to imitate the vision of the person with the designated CVD. If **both** options are set to `false`, nothing is recolored. If **both** options are set to `true`, the image will be recolored to imitate what the person with the designated CVD will see after the image is enhanced for their condition.

It is recommended to run the ishihara test scene as well for a more immersive experience:

```
>>> ./tracer -r 400 400 ../../tests/ishihara9.svg
```

Accomplishments

This effect is able to allow people with CVD to correctly distinguish commonly confused colors from each other when looking at the Ishihara test. This is testable since Drake and Henry have mild color blindness. Assuming `compensate` is set to `true` and `simulate` is set to `false`, the recoloring process would be as follows:

For each pixel:

1. Convert the RGB vector into a LMS (Long cone, Medium cone, Short cone) vector.

This categorizes colors into their respective wavelengths:

$$\begin{bmatrix} L_i \\ M_i \\ S_i \end{bmatrix} = \begin{bmatrix} 17.8824 & 43.5161 & 4.11935 \\ 3.45565 & 27.1554 & 3.86714 \\ 0.0299566 & 0.184309 & 1.46709 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

2. Apply CVD to LMS

- a. Protanopia:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 0 & 2.02344 & -2.52581 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} L_i \\ M_i \\ S_i \end{bmatrix}$$

- b. Deuteranopia:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0.494207 & 0 & 1.24827 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} L_i \\ M_i \\ S_i \end{bmatrix}$$

- c. Tritanopia:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0.395913 & 0.801109 & 0 \end{bmatrix} \begin{bmatrix} L_i \\ M_i \\ S_i \end{bmatrix}$$

3. Convert LMS back to RGB:

$$\begin{bmatrix} R_n \\ G_n \\ B_n \end{bmatrix} = \begin{bmatrix} 0.0809444479 & -0.130504409 & 0.116721066 \\ -0.0102485335 & 0.0540193266 & -0.113614708 \\ -0.000365296938 & -0.00412161469 & 0.693511405 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix}$$

4. Subtract this new RGB from the original RGB to get the **error** vector (the color that the person with CVD would have trouble seeing):

$$\text{RGB}_e = \text{RGB} - \text{RGB}_n$$

5. Calculate **fix** by shifting **error** towards the visible spectrum:

$$\begin{bmatrix} R_a \\ G_a \\ B_a \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0.7 & 1 & 0 \\ 0.7 & 0 & 1 \end{bmatrix} \begin{bmatrix} R_e \\ G_e \\ B_e \end{bmatrix}$$

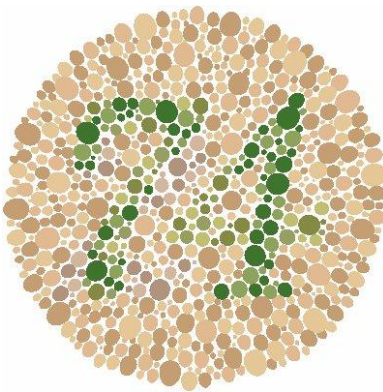
6. Add **fix** to original RGB to get final pixel color:

$$\text{RGB} += \text{RGB}_a$$

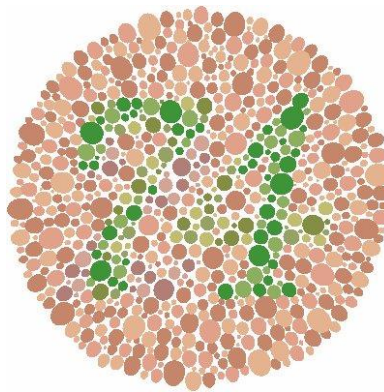
State-of-the-art

There are many types of daltonization methods available, each with their own strengths and weaknesses. Some notable methods include [Kotera's Method](#), [Machado's Method](#), [Hoffman's Method](#), and [Ebelin's Method](#). Methods with complex algorithms that aim for accuracy may not be suitable for real-time systems, for example. Ebelin's method was an interesting one, though the research paper provided is rather vague and it is difficult to follow through.

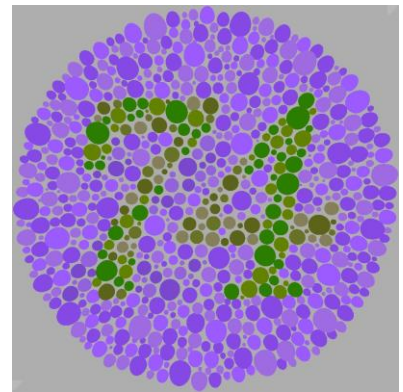
Results



Fixed for Protanopia



Fixed for Deuteranopia



Fixed for Tritanopia

What We Learned

All the daltonization methods serve in their own niche fields. The method implemented in this ray tracer is the most used one, and also possibly the simplest. Due to its simplicity, it can provide a general value to our final image output without requiring specific conditions.

Next Steps

Implement Ebelin's method, as it's the most applicable to what we're doing real-time wise.

Other Details

A custom parser was made from scratch to convert Ishihara SVG to OBJ/MTL files (`ishihara_svg_parser.h`). This parser currently only works for `ishihara9.svg`, because this parser makes spheres based on the `<circle>` tag. Other ishihara tests are either not in SVG form or use `<path>` instead of `<circle>`.

Blur

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

Edge Detect

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description

SSAO

Description

How to run

Description

Accomplishments

Description

State-of-the-art

Description

Results

Description

What We Learned

Description

Next Steps

Description

Other Details

Description